

Corso di crittografia A.A. 2023/2024

Documentazione progetto di un password cracker

Lorenzo Leoncini

May 1, 2025

1 Descrizione generale

Questo progetto implementa due programmi C che sfruttano il multithreading per effettuare un attacco brute force a un file **shadow**, utilizzando parole presenti in un dizionario. L'obiettivo è individuare password corrispondenti agli hash memorizzati nel file **shadow**, accelerando il processo grazie all'uso di più thread.

2 Struttura dei file

- **Makefile**: automatizza la compilazione dei due programmi.
- **dictionary.txt**: contiene una lista di parole comuni utilizzate come base per il cracking.
- **shadow**: simula un file **shadow** Unix con hash di password.
- **parallel_number.c**: programma che divide i tentativi di password tra i thread per ogni utente.
- **parallel_user.c**: programma che divide gli utenti tra i thread.

3 Funzionamento

3.1 `parallel_number.c`

- **Approccio**: Ogni thread esplora varianti di parole del dizionario per un utente alla volta.
- **Multithreading**: Ogni thread lavora su una porzione di numeri, per provare tutte le combinazioni tra 00 e 99.
- **Varianti**:
 - Prima lettera maiuscola.
 - Sostituzione di lettere comuni con numeri (es: o $\rightarrow$ 0, e $\rightarrow$ 3).
 - Aggiunta di suffissi numerici (da 00 a 99).

3.2 parallel_user.c

- **Approccio:** Divide gli utenti tra i thread; ogni thread processa un sottoinsieme di utenti, testando tutte le parole e varianti.
- **Multithreading:** Ogni thread lavora su utenti diversi per massimizzare la parallelizzazione quando ci sono molti utenti.
- **Varianti:** identiche a quelle di parallel_number.c.

4 Logica delle trasformazioni

Ogni parola del dizionario genera le seguenti varianti:

1. Originale (es: password)
2. Originale con numeri (es: pa55w0rd)
3. Capitalizzata (es: Password)
4. Capitalizzata + numeri (es: Pa55w0rd)

Per ciascuna variante, vengono aggiunti suffissi numerici da 00 a 99, ampliando ulteriormente lo spazio di ricerca.

5 Esecuzione

```
./pwd_guesserN <shadow_file> <dictionary_file> <num_threads>  
./pwd_guesserU <shadow_file> <dictionary_file> <num_threads>
```

Esempio:

```
./pwd_guesserN shadow dictionary.txt 4  
./pwd_guesserU shadow dictionary.txt 4
```

6 Multithreading

- Usa `pthread` per la gestione dei thread.
- Sincronizzazione:
 - In `parallel_number.c` viene usato `atomic_int` per segnalare agli altri thread quando la password è stata trovata.
 - In `parallel_user.c` un flag booleano locale gestisce la terminazione anticipata.

7 Dipendenze

- Libreria `crypt.h` (solitamente parte di `libcrypt` su Linux).
- Compilazione con `gcc` (supporto al multithreading e librerie standard).

8 Output

Ogni volta che una password viene trovata, il programma stampa:

```
Thread -> La password di utentex è:
```

Se non viene trovata alcuna password, il programma stampa invece:

```
Password non trovata per utentex
```

Alla fine stampa il tempo totale di esecuzione.

9 Problemi nella realizzazione

Durante lo sviluppo del progetto sono emerse diverse sfide, soprattutto in termini di ottimizzazione e gestione efficiente del multithreading.

Uno dei problemi principali è stato garantire che i thread rimanessero sempre occupati: nella versione che parallelizza sugli utenti (`parallel_user.c`), il problema non si presenta finché tutti i thread hanno un utente da processare, ma quando un thread termina prima degli altri, non è previsto un meccanismo per riassegnarlo ad altri compiti. Al contrario, nella versione che parallelizza sui numeri (`parallel_number.c`), questo problema non si verifica grazie a una sincronizzazione più fine che assicura il lavoro distribuito.

Un altro ostacolo iniziale riguardava le prestazioni: le letture venivano effettuate direttamente dai file `shadow` e `dictionary` in ogni iterazione, causando rallentamenti notevoli. La situazione è migliorata sensibilmente caricando il dizionario interamente in memoria all'avvio, riducendo la concorrenza tra thread e evitando aperture ripetute dei file.

10 Confronto tra i metodi e conclusioni

Un'analisi comparativa tra i due metodi implementati ha messo in evidenza alcune caratteristiche chiave. Se l'obiettivo principale è trovare le password di tutti gli utenti, la parallelizzazione sui numeri (`parallel_number.c`) risulta la più efficiente, poiché garantisce che i thread rimangano sempre attivi e sfrutta appieno la potenza di calcolo, mantenendo invariato il numero complessivo di operazioni di hashing, che costituiscono la parte più onerosa. Tuttavia, in scenari reali è più comune puntare a ottenere l'accesso a una singola password di un utente privilegiato, come l'amministratore (`root`), per compromettere il sistema. In questo contesto, la parallelizzazione sugli utenti (`parallel_user.c`) si rivela più vantaggiosa, perché riesce a identificare le password deboli più rapidamente.

11 Possibili miglioramenti

- Supporto per altri tipi di trasformazioni.
- Possibilità di riciclare i thread il cui lavoro è terminato, per mantenerli attivi e migliorare l'efficienza complessiva.
- Miglior gestione della memoria.
- Reporting più dettagliato (ad es. numero di tentativi effettuati).

Autore: Lorenzo Leoncini